

Internet of Things (IoT)

Lecture 7: IoT Protocols

Dr. Alyaa Hamza

Alyaa.hamza@sut.edu.eg

Dr. Nehal Anees

nehal.anees@sut.edu.eg

© Fall 2025/2026



IoT Course Topics

1. Introduction to IoT
2. IoT Architecture & Network Components
3. Smart Objects & IoT Devices
4. Communication Technologies in IoT
5. **IoT Protocol Stacks**
6. IoT Application Protocols & Data Formats
7. IoT Security Principles
8. Cloud & Fog Computing in IoT
9. Data Analytics in IoT
10. IoT Platforms & Middleware
11. Designing IoT Solutions

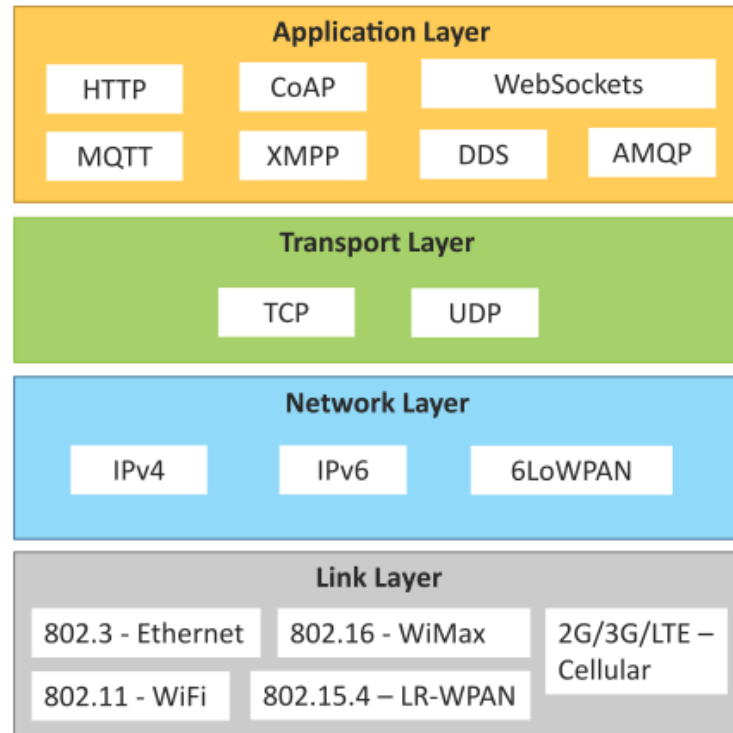


IoT Protocols



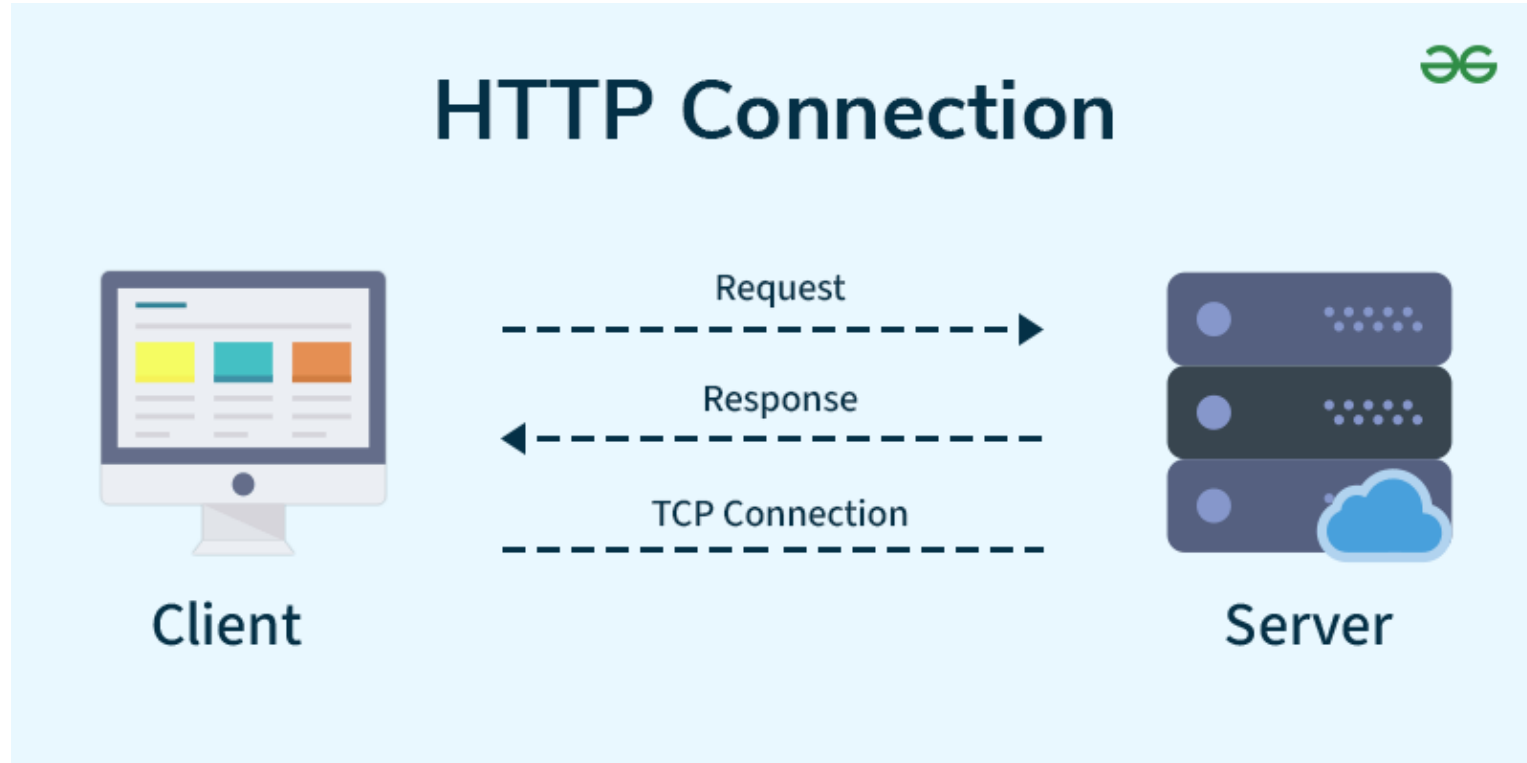
IoT Protocols

- Link Layer
 - 802.3 – Ethernet
 - 802.11 – WiFi
 - 802.16 – WiMax
 - 802.15.4 – LR-WPAN
 - 2G/3G/4G
- Network/Internet Layer
 - IPv4
 - IPv6
 - 6LoWPAN
- Transport Layer
 - TCP
 - UDP
- Application Layer
 - HTTP
 - CoAP
 - WebSocket
 - MQTT
 - XMPP
 - DDS
 - AMQP



HTTP Protocol

[Hypertext Transfer Protocol](#) (HTTP) is a protocol that is used for communication between client and server. This protocol is responsible for the response from the server side to the client response. It is a base protocol for the communication of web services.



Characteristics of HTTP

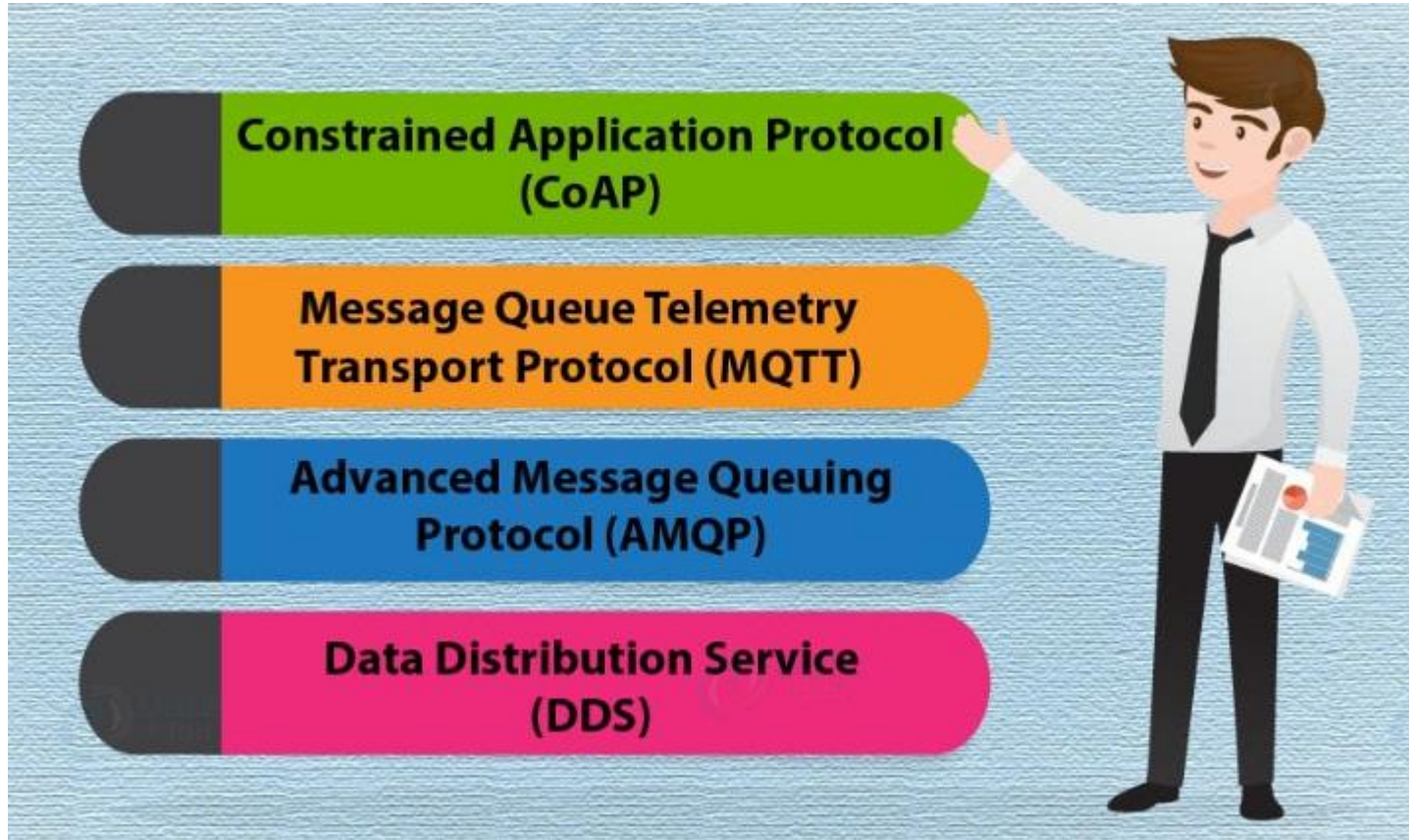
- Stateless:** Each request is independent, and the server doesn't retain previous interactions' information.
- Text-Based:** Messages are in plain text, making them readable and debuggable.
- Client-Server Model:** Follows a [client-server architecture](#) for requesting and serving resources.
- Request-Response:** Operates on a request-response cycle between clients and servers.
- Request Methods:** Supports various methods like GET, POST, PUT, DELETE for different actions on resources.

HTTP Protocol

Methods: HTTP supports several methods like : **GET, POST, DELETE , PUT**

- **GET** - The get method is used to retrieve resource information identified by the request URI.
- **POST** - The post method creates a new subordinate resource under the parent URI requested by it to the server.
- **DELETE** - The delete method deletes the resource identified by the requested URI .
- **PUT** - The PUT method updates or creates the resource identified by the request URI with the enclosed message body.

Constrained Application Protocol (CoAP)



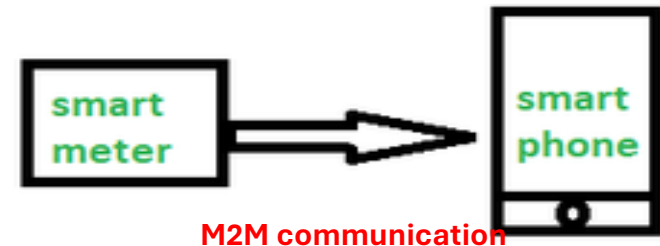
Constrained Application Protocol (CoAP)

The constrained application protocol is a web transfer protocol that works with constrained networks & nodes. Generally, these networks are used in [Internet of Things \(IoT\) applications](#), wherever devices have partial memory, power resources, and processors.

CoAP works as a type of HTTP for limited devices by allowing equipment like actuators or sensors to communicate on the Internet of Things.

This protocol is mainly designed for reliability within high congestion and low bandwidth through its low network overhead and less power consumption. CoAP in a network can continue to work with much congestion (or) limited connectivity wherever MQTT protocols fail to exchange data and converse efficiently.

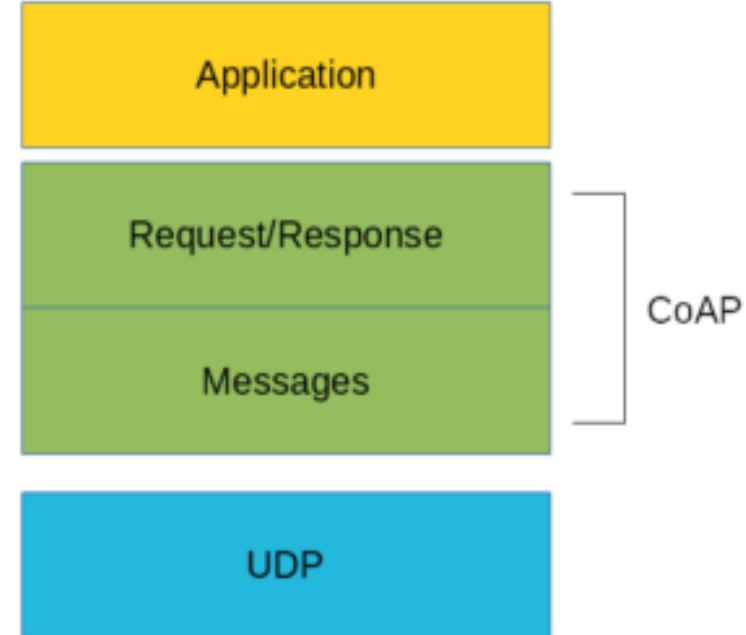
This protocol is very simple, efficient, and has low overhead for machine-based communication. CoAP is mainly designed to connect to the IoT even through limited networks through



CoAP Architecture

The architecture of CoAP is as shown below:

- There are two different layers that make CoAP protocol: Messages and Request/Response.
- The Messages layer deals with UDP and with asynchronous messages.
- ✓ **Note : Asynchronous Messaging:** CoAP messages are asynchronous because it uses the User Datagram Protocol (UDP). Unlike TCP-based protocols, CoAP does not require acknowledgments for every message, which helps conserve energy in resource-constrained devices.
- The Request/Response layer manages request /response interaction based on request/response messages.



Asynchronous Message Exchanges

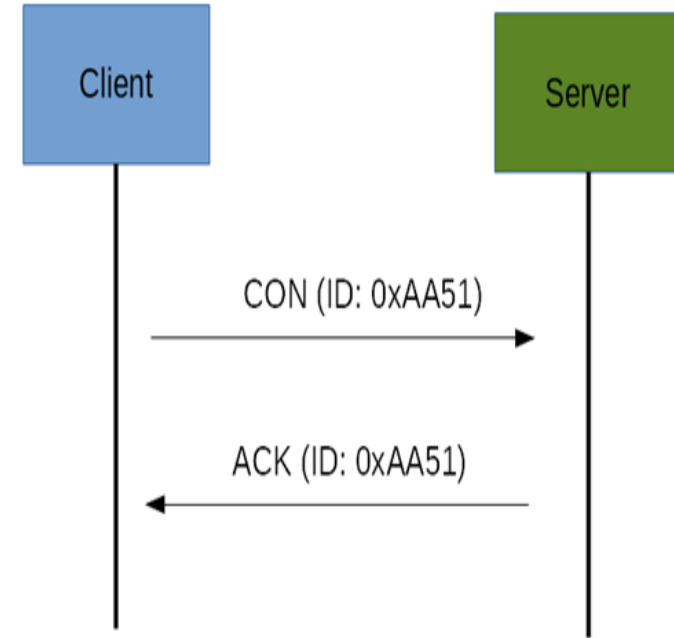
- CoAP protocol simply supports asynchronous message connections, which is essential for IoT networks wherever different devices may not be available or connected always.
- The device can send a request to another device with asynchronous message exchanges, and after that, it continues with different tasks exclusive of waiting for a reply. This response can be processed whenever it appears even if postponed.
- This feature utilizes a message identifier within every CoAP protocol message to allow a device to match responses with requests. So this in combination with the capacity to retransmit lost messages ensures a high range of reliability within exchanges of messages.

Messaging in CoAP

CoAP protocol uses two kinds of messages:

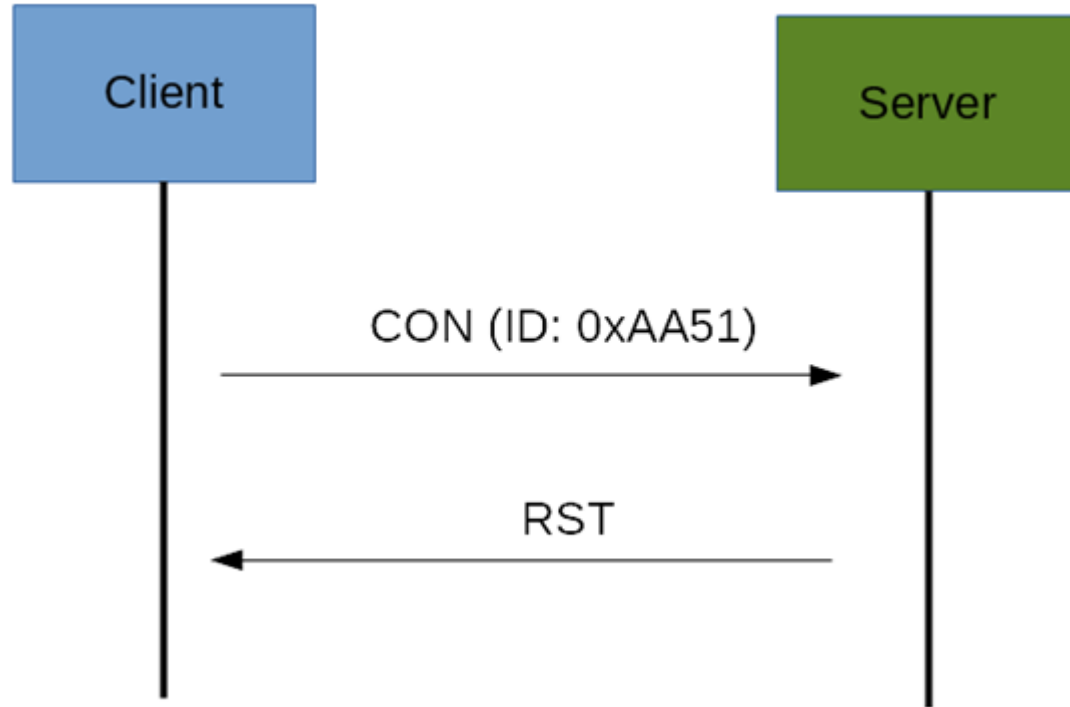
- 1) Confirmable message,
- 2) Non-confirmable message

A confirmable message is a reliable message. Using this kind of message, the client can be sure that the message will arrive at the server. A Confirmable message is sent again and again until the other party sends an acknowledge message (ACK) as shown below. The ACK message contains the same ID of the confirmable message



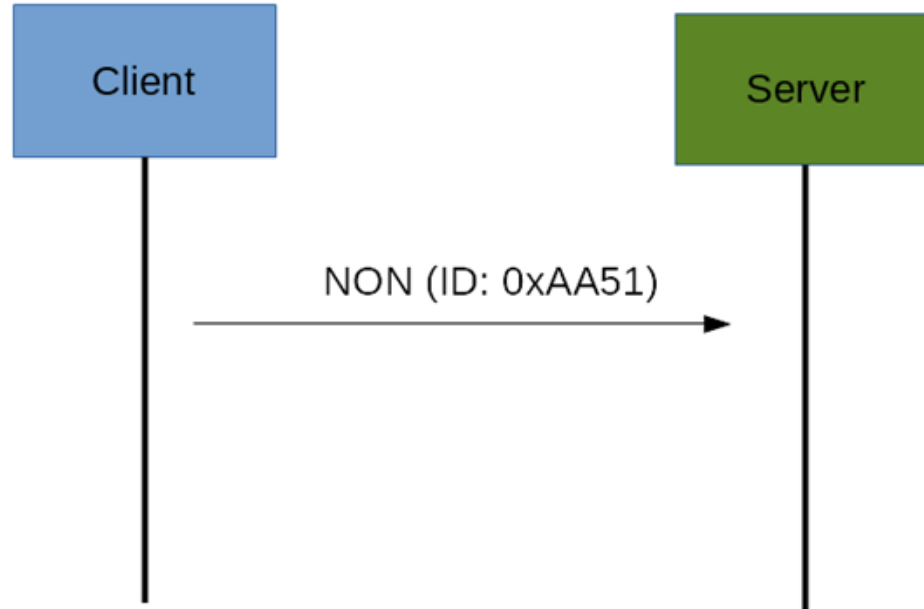
confirmable messages

If the server has troubles managing the incoming request, it can send back a Rest message (RST) instead of the Acknowledge message (ACK) as shown below.



Non-confirmable (NON) messages

- The other message category is the Non-confirmable (NON) messages.
 - These are messages that don't require an Acknowledge by the server. They are unreliable messages or in other words messages that do not contain critical information that must be delivered to the server. To this category belongs messages that contain values read from sensors. Even if these messages are unreliable, they have a unique ID as shown below.



Thank You